

Student

I've always just written everything in one file, or copied code between files with copy-paste. My senior told me I need to learn how Python's module and package system actually works. Where do I start?

ChatGPT • AI Assistant

CONCEPT CARD

Start with the vocabulary. In Python:

- A **module** is any single `.py` file. `utils.py` is a module. `math` is a module.
- A **package** is a directory that contains Python modules — and traditionally a special file called `__init__.py` that marks it as a package.
- A **namespace package** (Python 3.3+) is a package WITHOUT `__init__.py` — we'll cover that too.

When you write `import math` or `from utils import helper`, Python runs a precise lookup algorithm involving `sys.path`, a module cache, and file system searches. Understanding this process will eliminate entire categories of `ModuleNotFoundError` and `ImportError` bugs that frustrate developers for hours.

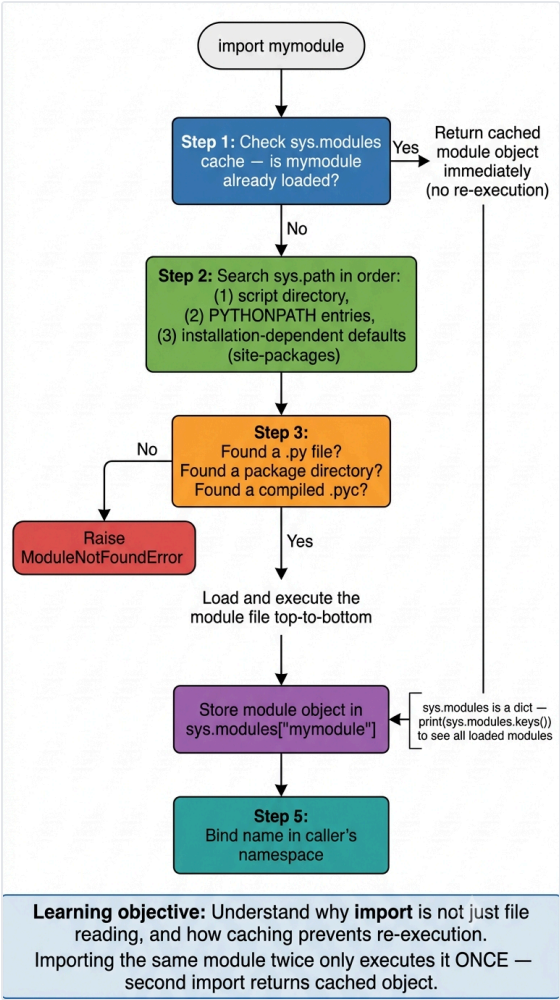
Student

Why does it even matter how import works internally? It just works, right?

ChatGPT • AI Assistant

💡 ANALOGY

Knowing how `import` works is like knowing how a GPS finds routes. You can follow turn-by-turn directions without understanding it — until the GPS sends you into a lake. When you get a `ModuleNotFoundError` on a machine that's not yours, or a `circular import` error on a module you know exists, or a module that imports fine locally but fails in Docker — you need to understand the GPS algorithm to fix it. The import system is Python's GPS for finding code.



Python Import System — Full Lookup Pipeline

Module vs Package vs Namespace Package — Key Differences			
Type	Structure	<code>__init__.py</code> ?	Use Case
Module	Single <code>.py</code> file	N/A	Small utilities, standalone scripts
Regular Package	Directory + <code>__init__.py</code>	Required (can be empty)	Most libraries and application code
Namespace Package	Directory only (no <code>__init__.py</code>)	Absent	Splitting a package across multiple directories/repos
Sub-package	Nested directory + <code>__init__.py</code> inside a package	Required	Logical grouping within a large package

Type	Structure	__init__.py?	Use Case
Built-in module	Part of CPython interpreter	N/A	os, sys, math — always available

HOW IMPORT WORKS — SYS.PATH, SYS.MODULES, AND THE MODULE CACHE

ChatGPT • AI Assistant

A horizontal search-order diagram. On the left, a list of sys.path entries numbered [0] to [3]. Entry [0] (red background): 'Script directory — SEARCHED FIRST — shadowing risk here'. Entry [1] (yellow): 'PYTHONPATH env var entries'. Entry [2] (green): 'Standard library (/usr/lib/python3.x)'. Entry [3] (blue): 'site-packages (/usr/lib/python3.x/site-packages — third-party installs)'. From each entry, a horizontal arrow points right to a 'Found?' diamond. All 'No' branches link to the next entry. The final 'No' branch leads to 'ModuleNotFoundError'. The first 'Yes' branch leads to 'Load module — stop searching'. On the right, a warning box with example: 'You have math.py in your project → import math loads YOURS, not stdlib. sys.path[0] is checked before sys.path[2].' Learning objective: Internalize the search order and understand why script directory is first and why that creates shadowing risk.

sys.path Search Order and Module Shadowing Risk

Student

You mentioned `sys.path` — what is that exactly?

ChatGPT • AI Assistant

CONCEPT CARD

`sys.path` is a Python list of directory strings that Python searches — **in order** — when you write `import something`. It is Python's module search path. Think of it as a list of locations Python checks, one by one, until it finds a match or runs out of places to look.

EXAMPLE CARD

Example 1 — Basic: Inspecting sys.path live

python — Code Example

```
import sys
```

```
# Print your current search path
for i, path in enumerate(sys.path):
    print(f'[{i}] {path}')

# Typical output:
# [0] (empty string = current working directory)
# [1] /home/srikanth/.local/lib/python3.11/site-packages
# [2] /usr/lib/python3.11
# [3] /usr/lib/python3.11/lib-dynload
# ...

# sys.path[0] is always the script's own directory
# (or '' meaning the cwd when running interactively)
```

Student

So if I put a file called `math.py` in my project folder, it would shadow Python's built-in `math` module?

ChatGPT • AI Assistant

Exactly — and this is one of the most common beginner pitfalls. Because `sys.path[0]` is the script's directory (searched first), any local file with the same name as a stdlib module will shadow it. Name your files `math.py`, `random.py`, `string.py`, or `os.py` and suddenly `import math` loads YOUR file, not Python's. This causes baffling errors.

❗ COMMON PITFALL / MISTAKE

✗ Incorrect — naming your file after a stdlib module:

code — Code Example

```
project/
  math.py          ← YOUR file
  calculator.py    ← does 'import math' — loads YOUR math.py!
```

python — Code Example

```
# math.py (your accidental shadow)
def add(a, b): return a + b

# calculator.py
```

```
import math
result = math.sqrt(16) # AttributeError: module 'math' has no
                        # attribute 'sqrt'
                        # Because 'math' is YOUR file, not the stdlib!
```

✓ Correct — use project-specific names:

code — Code Example

```
project/
  arithmetic.py    ← safe name
  calculator.py    ← 'import arithmetic' works fine
```

EXAMPLE CARD

Example 2 — Intermediate: The sys.modules cache — why importing twice is free

python — Code Example

```
import sys
import time

# First import — file is read and executed
start = time.perf_counter()
import json
print(f'First import: {(time.perf_counter()-start)*1000:.3f}ms')

# Second import — returns from cache instantly
start = time.perf_counter()
import json # Python does NOT re-read the file
print(f'Second import: {(time.perf_counter()-start)*1000:.3f}ms')

# Check the cache:
print('json' in sys.modules)      # True
print(type(sys.modules['json']))  # <class 'module'>

# Both 'json' names point to the SAME object:
import json as json1
import json as json2
print(json1 is json2) # True — same object from cache
```

Student

What does this mean practically? Why does it matter that the module only executes once?

ChatGPT • AI Assistant

It means **module-level code is initialization code** — it runs once when first imported, then never again. This is why module-level singletons (database connections, config objects, loggers) work: you initialize them once at module load time, and every other import gets the same already-initialized object from the cache.

But it also means: if a module has side effects (prints something, connects to a DB, modifies global state), those effects happen exactly once — the first time it is imported. Subsequent imports are silent.

EXAMPLE CARD

Example 3 — Practical: Module-level initialization pattern

python — Code Example

```
# config.py
import os
print('[CONFIG] Loading configuration...') # This prints ONCE only

DATABASE_URL = os.getenv('DATABASE_URL', 'sqlite:///dev.db')
DEBUG = os.getenv('DEBUG', 'false').lower() == 'true'
SECRET_KEY = os.getenv('SECRET_KEY', 'dev-only-secret')

# app.py
import config # '[CONFIG] Loading configuration...' printed HERE
print(config.DATABASE_URL)

# models.py
import config # NOTHING printed – config already in sys.modules
print(config.DEBUG) # Same config object as app.py

# views.py
import config # NOTHING printed – same cached object
```

Student

Can I add to `sys.path` myself to import from a non-standard location?

Yes — but it is rarely the right answer. For production code, use proper package installation (`pip install -e .`) or set `PYTHONPATH` . Modifying `sys.path` at runtime is a code smell in library code because it has **global side effects** — every subsequent import in the process is affected. In scripts and quick hacks it's acceptable.

python — Code Example

```
import sys
sys.path.insert(0, '/path/to/my/custom/modules') # Adds to the FRONT
import my_custom_module # Now found
# WARNING: modifying sys.path affects ALL imports in this process!
```

INTERVIEW CARD

- **Frequency:** Medium-high — asked in Python backend and DevOps roles
- **Question:** 'What is `sys.path` and in what order does Python search it when you write `import foo` ?'
- **Expected Answer:** `sys.path` is a list of directory paths. Python searches them in order: (1) the script's own directory (or `''` for interactive), (2) any directories from `PYTHONPATH` env var, (3) installation defaults including `site-packages` . First match wins. If nothing found, `ModuleNotFoundError` .
- **Follow-up:** 'What is `sys.modules` and why does it exist?'

EXERCISE

Open a Python REPL and investigate your own environment:

1. Print `sys.path` and identify which entry points to your virtual environment's `site-packages`
2. Import any module (e.g., `json`), then check `sys.modules['json']` — what does it show?
3. Try `import sys; del sys.modules['json']; import json` — what happens? Does `json` re-execute?
4. Create a file called `os.py` in your current directory with one line:
`print('SHADOW!')` . Then open Python in that directory and `import os` . What happens?

AI DEEP DIVE PROMPT

Copy this into Claude or ChatGPT:

"Act as a Python internals expert. Teach me Python's import system step by step through 10 hands-on experiments. Cover: sys.path manipulation, sys.modules inspection, module shadowing, importlib.import_module(), lazy imports, the .pyc cache, reload(), import hooks, and namespace packages. After each experiment: (1) ask me to predict what happens, (2) show the result, (3) explain the underlying mechanism, (4) give one interview question about it."

🔗 PRACTICE QUIZ

Concept Questions:

1. What does `sys.path[0]` typically contain, and why is it dangerous to have a file named `os.py` in that location?
2. If you `import json` ten times in different files of a project, how many times does `json.py` actually get executed?
3. What is stored as the value in `sys.modules['math']` after `import math` ?

Output Prediction Questions:

1. What does this print?

python – Code Example

```
import sys
import json
print('json' in sys.modules)
print(id(sys.modules['json']) == id(json))
```

1. You have a file `string.py` in your project root with `MY_VAR = 42` . In the same directory you run:

python – Code Example

```
import string
print(string.MY_VAR)
```

What is the output?

Debugging Question:

1. A developer's project has this structure and gets `ModuleNotFoundError: No module named 'helpers'` :

code – Code Example

```
project/
  src/
    main.py      ← runs this
    helpers.py
```

python – Code Example

```
# main.py
import helpers # ModuleNotFoundError!
```

What are two ways to fix this?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. `sys.path[0]` is the directory of the currently running script (or `'.'` for the cwd in interactive mode). A file named `os.py` there would shadow Python's stdlib `os` module — any `import os` would load your file instead, breaking almost everything.
2. Exactly once — on first import. All subsequent `import json` calls return the cached module object from `sys.modules`.
3. The module object — a `<module 'math' ...>` instance. It is the live Python module object with all attributes (`math.sqrt`, `math.pi`, etc.) accessible on it.
4. Prints `True` then `True` — `json` is in `sys.modules`, and `sys.modules['json']` is the exact same object as the `json` name bound in your namespace.
5. Prints `42` — because `sys.path[0]` is the project root, your `string.py` shadows the stdlib `string` module. `string.MY_VAR` returns `42` instead of the normal stdlib string constants.
6. Fix 1: Run Python from the `src/` directory so `src/` is in `sys.path[0]`. Fix 2: Add an `__init__.py` to `src/` and run from `project/` using `python -m src.main`, or install the package with `pip install -e .` so Python's package system handles the path.

❓ CHECKPOINT QUESTION

A colleague says 'importing the same module in 20 different files is slow.' Is this true or false? Explain using `sys.modules`.

ABSOLUTE VS RELATIVE IMPORTS — WHEN TO USE EACH

ChatGPT • AI Assistant

A file tree diagram on the left showing the package structure: `data_pipeline/` (root) → `config.py`, `utils/` (→ `validators.py`, `parsers.py`), `stages/` (→ `transform.py`, `extract.py`, `load.py`). On the right, three import resolution arrows originating from `transform.py`. Arrow 1 (blue, labeled 'Absolute: from `data_pipeline.config` import `SCHEMA`'): starts at `sys.path` root, traces down through `data_pipeline/` → `config.py`. Arrow 2 (green, labeled 'Relative: from `.` import `load`'): starts at current location (`stages/`), traces horizontally to `load.py` in the same directory. Arrow 3 (orange, labeled 'Relative: from `..utils` import `parsers`'): starts at `stages/`, goes UP one level to `data_pipeline/`, then DOWN into `utils/parsers.py`. Each arrow is color-coded with a legend. A note box: 'Absolute = navigate from `sys.path` root. Relative = navigate from current package.' Bottom warning: 'Relative imports require `__package__` to be set — fails in directly-run scripts.' Learning objective: Visually understand the navigation difference between absolute and relative imports.

Absolute vs Relative Imports — Navigation Model

Student

I've seen `from . import utils` with a dot in some code. What does the dot mean?

ChatGPT • AI Assistant

CONCEPT CARD

The dot in `from . import utils` is a **relative import**. It means 'import `utils` relative to the current package — not from anywhere on `sys.path`.' One dot = current package. Two dots (`..`) = parent package. Three dots = grandparent package.

Absolute imports (no dots) search `sys.path` from scratch every time. **Relative imports** (with dots) navigate the package tree starting from where the current module lives.

EXAMPLE CARD

Example 1 — Basic: Absolute vs relative side by side

code — Code Example

```
mypackage/
  __init__.py
  utils.py
  models.py
  api/
    __init__.py
```

```
routes.py
helpers.py
```

python — Code Example

```
# Inside mypackage/models.py

# ABSOLUTE import — searches sys.path, finds mypackage/utils.py
from mypackage import utils          # Full path from sys.path root
from mypackage.utils import validate # Import specific name

# RELATIVE import — navigates from current package (mypackage)
from . import utils                  # . = mypackage (current
package)
from .utils import validate          # Same thing, specific name
```

python — Code Example

```
# Inside mypackage/api/routes.py

# Absolute — always works from sys.path
from mypackage import utils          # Finds mypackage/utils.py
from mypackage.api import helpers    # Finds mypackage/api/helpers.py

# Relative — navigates from mypackage/api/
from . import helpers                # . = mypackage/api (current)
from .. import utils                  # .. = mypackage (parent)
from ..models import User             # .. = mypackage, then models.py
```

Student

When should I use relative imports instead of absolute?

ChatGPT • AI Assistant

The Python community guidance (PEP 8) is to **prefer absolute imports** for clarity — they always tell you the full path to what you are importing, making the code readable anywhere. Use relative imports when:

- You are inside a package and want to import a sibling or parent module **without hardcoding the package name** — this makes the package easier to rename or reorganize

- You are writing reusable library code that should not depend on the package being installed with a specific name

Never use relative imports in scripts (files run directly with `python script.py`) — they only work inside packages.

EXAMPLE CARD

Example 2 — Intermediate: Why relative imports fail in scripts

python — Code Example

```
# mypackage/utils.py
from . import models  # This is a relative import

# If you run: python mypackage/utils.py
# Error: ImportError: attempted relative import with no known parent
package
# Because Python treats the directly-run file as __main__ — not as
part of a package
# __main__ has no parent package to resolve . from

# Fix: run as a module, not as a script
# python -m mypackage.utils  ← This works — Python knows the package
context
```

Student

That's confusing. So the same file works differently depending on HOW I run it?

ChatGPT • AI Assistant

Exactly — and this surprises almost every developer the first time. When Python runs a file as a script (`python file.py`), it sets `__name__ = '__main__'` and `__package__ = None`. Relative imports need `__package__` to be set to resolve the `.` — without it, Python has no idea what package the current module belongs to. Running with `python -m package.module` sets `__package__` correctly.

EXAMPLE CARD

Example 3 — Practical: A package with both absolute and relative imports

code — Code Example

```

data_pipeline/
  __init__.py
  config.py
  utils/
    __init__.py
    validators.py
    parsers.py
  stages/
    __init__.py
    extract.py
    transform.py
    load.py

```

python – Code Example

```

# data_pipeline/stages/transform.py

# Absolute imports – explicit, always readable
from data_pipeline.config import SCHEMA          # Cross-package: go
to root
from data_pipeline.utils.validators import validate_record

# Relative imports – useful for sibling stages
from . import load                                # Sibling in stages/
from ..utils import parsers                       # Go up to
data_pipeline/, then utils/
from ..utils.parsers import parse_date            # Specific name from
parent's subpackage

# Mixing is fine – use whichever is clearest for each import

```

❗ COMMON PITFALL / MISTAKE

✗ Incorrect — relative import in a directly-run script:

python – Code Example

```

# run as: python data_pipeline/stages/transform.py
from ..utils import parsers  # ImportError! No known parent package.
                             # __package__ is None when run as
script.

```

✓ Correct — run as a module OR use absolute imports in scripts:

bash — Code Example

```
# Option 1: Run as module (preserves package context)
python -m data_pipeline.stages.transform

# Option 2: Use absolute imports everywhere (safest)
```

python — Code Example

```
from data_pipeline.utils import parsers # Works regardless of how you
run it
```

 INTERVIEW CARD

- **Frequency:** Medium — asked in Python backend and data engineering roles
- **Question:** 'What is the difference between `from . import utils` and `from mypackage import utils` ? When does each fail?'
- **Expected Answer:** The first is a relative import — resolves relative to the current package using `__package__` . Fails when the module is run directly as a script (`__package__` is None). The second is an absolute import — searches `sys.path` . Fails when `mypackage` is not on `sys.path` . Absolute imports are preferred by PEP 8 for clarity.
- **Follow-up:** 'What does `..` mean in `from .. import config` ?'

 EXERCISE

Given this package structure:

code — Code Example

```
shop/
  __init__.py
  models.py      ← defines class Product, class Order
  services/
    __init__.py
    pricing.py   ← needs Product from models.py
    inventory.py ← needs Product from models.py
  api/
    __init__.py
    routes.py    ← needs Order from models.py AND pricing from
services/
```

Write the correct import statements for each file using:

1. Absolute imports only
2. Relative imports only
3. Identify which file would break if run directly with `python shop/api/routes.py` when using relative imports

🔍 PRACTICE QUIZ

Concept Questions:

1. What does a single dot (`.`) refer to in `from . import helpers` ?
2. Why do relative imports fail when a module is run directly with `python file.py` ?
3. What PEP recommends for import style, and what does it say about relative vs absolute?

Output Prediction Questions:

1. Given the `data_pipeline` structure above, what does `from ..utils import parsers` import when executed inside `stages/transform.py` ?
2. What is the value of `__package__` inside `transform.py` when run as `python -m data_pipeline.stages.transform` ?

Debugging Question:

1. A developer gets `ImportError: attempted relative import beyond top-level package` with this import in `mypackage/utils.py` :

python – Code Example

```
from ... import something
```

What is wrong?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. The single dot `.` refers to the current package — the package that contains the current module file. If the current file is `mypackage/api/routes.py` , then `.` refers to `mypackage/api` .
2. When run directly, Python sets `__name__ = '__main__'` and `__package__ = None` . Relative imports use `__package__` to resolve the parent package chain.

With `__package__ = None`, Python cannot resolve what `.` refers to.

3. PEP 8 says: 'Absolute imports are recommended, as they are more readable and tend to be better behaved.' Relative imports are acceptable for intra-package imports but should be used sparingly.
4. It imports the `parsers` module from `data_pipeline/utils/` — going up one level (`..`) to `data_pipeline/`, then into `utils/`.
5. `__package__` is `'data_pipeline.stages'` — the full dotted package path of the containing package.
6. Three dots (`...`) would try to go THREE levels up from `mypackage/utils.py`: `mypackage` → parent of `mypackage` → grandparent. If `mypackage` is already the top-level package on `sys.path`, there is no level above it to import from. Fix: use two dots maximum from the deepest sub-package, or switch to absolute imports.

🔗 CHECKPOINT QUESTION

You are renaming `mypackage` to `core`. Which import style — absolute or relative — requires fewer file edits across the entire package, and why?

BUILDING A PACKAGE — `__INIT__.PY`, PUBLIC API, AND SUBMODULES

ChatGPT • AI Assistant

A before/after diagram. Left panel (labeled 'Before: Monolith'): One file box 'pipeline.py' (red border) with internal sections listed: CONFIG, NETWORK, VALIDATION, TRANSFORM, MAIN — all inside one rectangle. Labels: '300+ lines', 'Hard to test', 'Hard to reuse', 'Hard to navigate'. Right panel (labeled 'After: Package'): A tree structure. Root box 'datapipeline/' (green border) with children: '__init__.py' (public API, `__all__`, `__version__`), 'config.py' (constants), 'exceptions.py' (custom errors), 'network/' (with `client.py` inside), 'validation/' (with `users.py`, `orders.py` inside), 'transform/' (with `users.py`, `orders.py` inside), 'pipeline.py' (orchestrator). Arrows from each child box show: 'Testable independently', 'Importable separately', 'Clear responsibility'. Center connecting arrow labeled 'Refactor'. Bottom note: '`__init__.py` exposes the public surface — callers are shielded from internal reorganization.' Learning objective: See how packages add maintainability without changing any logic.

Package Refactoring — Monolith to Layered Package Structure

Student

I have a 300-line Python script that's getting hard to manage. How do I convert it into a proper package?

ChatGPT • AI Assistant

⊕ REAL WORLD APPLICATION

This is the most practical skill in this section. Almost every project eventually outgrows a single file. Let me show you the step-by-step process — we'll take a monolithic `pipeline.py` script and restructure it into a proper package.

EXAMPLE CARD

Example 1 — Basic: The monolith before refactoring

python — Code Example

```
# pipeline.py — 300 lines, hard to maintain
import os
import json
import requests

# — CONFIG —————
API_URL = os.getenv('API_URL', 'https://api.example.com')
TIMEOUT = 10

# — NETWORK FUNCTIONS —————
def fetch_records(endpoint, params=None):
    """Fetch records from the API."""
    response = requests.get(f'{API_URL}/{endpoint}', params=params,
                             timeout=TIMEOUT)
    response.raise_for_status()
    return response.json()

def post_record(endpoint, data):
    """Post a record to the API."""
    response = requests.post(f'{API_URL}/{endpoint}', json=data,
                              timeout=TIMEOUT)
    response.raise_for_status()
    return response.json()

# — VALIDATION FUNCTIONS —————
def validate_user(record):
    required = ['id', 'name', 'email']
    for field in required:
        if field not in record:
            raise ValueError(f'Missing field: {field}')
    return True

def validate_order(record):
    if record.get('total', 0) <= 0:
```

```

        raise ValueError('Order total must be positive')
    return True

# — TRANSFORMATION FUNCTIONS —————
def normalize_user(record):
    return {'id': int(record['id']), 'name': record['name'].strip(),
            'email': record['email'].lower()}

def normalize_order(record):
    return {'id': int(record['id']), 'total':
round(float(record['total']), 2),
            'user_id': int(record['user_id'])}

# — MAIN —————
def run_pipeline():
    users = fetch_records('users')
    for user in users:
        validate_user(user)
        normalized = normalize_user(user)
        post_record('processed/users', normalized)

if __name__ == '__main__':
    run_pipeline()

```

EXAMPLE CARD**Example 2 — Intermediate: The refactored package structure**

code — Code Example

datapipeline/	← package root directory
__init__.py	← marks it as a package; defines public API
config.py	← all configuration constants
exceptions.py	← custom exception classes
network/	
__init__.py	
client.py	← HTTP functions (fetch_records,
post_record)	
validation/	
__init__.py	
users.py	← validate_user
orders.py	← validate_order
transform/	
__init__.py	
users.py	← normalize_user

orders.py	← normalize_order
pipeline.py	← run_pipeline() – orchestrates everything

python – Code Example

```
# datapipeline/config.py
import os

API_URL = os.getenv('API_URL', 'https://api.example.com')
TIMEOUT = int(os.getenv('TIMEOUT', '10'))

# datapipeline/network/client.py
import requests
from datapipeline.config import API_URL, TIMEOUT
from datapipeline.exceptions import NetworkError

def fetch_records(endpoint, params=None):
    try:
        response = requests.get(f'{API_URL}/{endpoint}',
                                params=params, timeout=TIMEOUT)
        response.raise_for_status()
        return response.json()
    except requests.RequestException as e:
        raise NetworkError(f'Failed to fetch {endpoint}') from e

# datapipeline/validation/users.py
from datapipeline.exceptions import SchemaValidationError

def validate_user(record):
    required = ['id', 'name', 'email']
    for field in required:
        if field not in record:
            raise SchemaValidationError(
                f'Missing field: {field}',
                field_name=field
            )

# datapipeline/transform/users.py
def normalize_user(record):
    return {
        'id': int(record['id']),
        'name': record['name'].strip(),
        'email': record['email'].lower()
    }
```

What goes in `__init__.py` ? Is it always empty?

ChatGPT • AI Assistant

CONCEPT CARD

`__init__.py` serves two purposes:

1. **Marks the directory as a package** — without it (in Python 2, and for regular packages in Python 3), the directory is not importable
2. **Defines the public API** — what you put in `__init__.py` is what users of your package get when they `import datapipeline`

Best practice: keep `__init__.py` lean. Its job is to expose the package's public surface — re-export the things callers need, and keep internal structure hidden.

EXAMPLE CARD

Example 3 — Practical: Designing a clean `init.py`

python — Code Example

```
# datapipeline/__init__.py
"""
datapipeline — ETL pipeline for user and order data.

Public API:
    run_pipeline() — run the full pipeline
    NetworkError — raised on API failures
    ValidationError — raised on bad data
"""

# Import what callers need — hide internal structure
from datapipeline.pipeline import run_pipeline      # Main entry point
from datapipeline.exceptions import (
    DataPipelineError,
    NetworkError,
    SchemaValidationError,
    DataQualityError
)

# Explicitly declare the public API
# Anything NOT in __all__ is considered private/internal
__all__ = [
    'run_pipeline',
```

```

    'DataPipelineError',
    'NetworkError',
    'SchemaValidationError',
    'DataQualityError',
]

# Package metadata
__version__ = '1.2.0'
__author__ = 'Data Engineering Team'

```

python – Code Example

```

# Now callers can use the clean public API:
from datapipeline import run_pipeline      # Works – exposed in
__init__.py
from datapipeline import NetworkError      # Works

# Internal modules are still accessible but considered private:
from datapipeline.network.client import fetch_records # Works but
discouraged
# If you rename client.py to http_client.py, this breaks.
# But callers using only __init__.py are shielded from that change.

```

Student

What does `__all__` actually do?

ChatGPT • AI Assistant

`__all__` is a list of strings that defines what is exported when someone does `from datapipeline import *`. Without `__all__`, `import *` imports everything that doesn't start with an underscore. With `__all__`, only the listed names are exported. It is also a strong **documentation signal** to other developers: 'These are the things I officially support. Everything else is internal and may change.'

❗ COMMON PITFALL / MISTAKE

✗ Incorrect — everything in `init.py` as a dump:

python – Code Example

```

# __init__.py – BAD: imports everything from everywhere
from .network.client import *

```

```

from .validation.users import *
from .validation.orders import *
from .transform.users import *
from .transform.orders import *
# Now `from datapipeline import *` brings in 30+ names
# Namespace is polluted; impossible to know where anything comes from

```

✓ Correct — curated public API:

python — Code Example

```

# __init__.py — GOOD: explicit, minimal, documented
from .pipeline import run_pipeline
from .exceptions import DataPipelineError, NetworkError
__all__ = ['run_pipeline', 'DataPipelineError', 'NetworkError']

```

INTERVIEW CARD

- **Frequency:** High for mid-level and senior Python roles
- **Question:** 'What is the purpose of `__init__.py` in a Python package? What should and should not go in it?'
- **Expected Answer:** `__init__.py` marks the directory as a package and runs when the package is first imported. It should contain: public API re-exports, `__all__` definition, `__version__`. It should NOT contain: heavy business logic, large amounts of code, or wildcard star imports from submodules (namespace pollution).
- **Follow-up:** 'What is `__all__` and what happens during `from package import *` with and without it?'

EXERCISE

Take this flat script and restructure it as a package:

python — Code Example

```

# report_generator.py (200 lines — restructure this)
import csv, json, smtplib

SMTP_HOST = 'smtp.company.com'

def read_csv(path): ...
def read_json(path): ...
def filter_records(records, min_value): ...
def aggregate_by_category(records): ...

```

```
def format_html_report(data): ...
def format_csv_report(data): ...
def send_email(to, subject, body): ...
def run_report(input_path, output_format, email_to): ...
```

Create a package structure with at least 3 submodules. Write the `__init__.py` with a proper `__all__`. Show the import statements inside each submodule.

🔗 PRACTICE QUIZ

Concept Questions:

1. What are the two purposes of `__init__.py` in a Python package?
2. What does `__all__` control, and where does it have an effect?
3. Why is it bad practice to put heavy logic in `__init__.py`?

Output Prediction Questions:

1. If `datapipeline/__init__.py` contains `from .pipeline import run_pipeline`, what happens when you do `import datapipeline`?
2. Given `__all__ = ['run_pipeline']` in `__init__.py`, what does `from datapipeline import *` import?

Debugging Question:

1. A developer creates this package but gets `ImportError: cannot import name 'run_pipeline' from 'datapipeline'`:

code — Code Example

```
datapipeline/
  __init__.py  ← empty
  pipeline.py  ← has def run_pipeline(): ...

from datapipeline import run_pipeline  # ImportError!
```

What is wrong and what are two fixes?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

- (1) Marks the directory as a Python package, making it importable. (2) Runs initialization code and defines the public API surface when the package is first imported.
- `__all__` controls what names are exported by `from package import *`. It is also a convention signal to developers about the public vs internal API, but it does not prevent direct imports of non-listed names.
- Heavy logic in `__init__.py` slows down every import of the package (since `__init__.py` runs on first import), creates tight coupling, and makes the package harder to test and maintain.
- Python executes `datapipeline/__init__.py`, which imports `run_pipeline` from `datapipeline/pipeline.py` and binds it in the `datapipeline` namespace. `run_pipeline` is now accessible as `datapipeline.run_pipeline`.
- Only `run_pipeline` is imported into the caller's namespace — nothing else, even if other names exist in the package.
- Problem: `__init__.py` is empty — `run_pipeline` is defined in `pipeline.py` but not exposed in `__init__.py`. Fix 1: Add `from .pipeline import run_pipeline` to `__init__.py`. Fix 2: Change the import to `from datapipeline.pipeline import run_pipeline` (direct submodule access — bypasses `__init__.py`).

🔍 CHECKPOINT QUESTION

You rename `datapipeline/network/client.py` to `datapipeline/network/http.py`. Which callers break: those using `from datapipeline import NetworkError` (via `__init__.py`), or those using `from datapipeline.network.client import fetch_records` (direct submodule access)?

NAMESPACE PACKAGES, CIRCULAR IMPORTS, AND FIXING BROKEN IMPORT CHAINS

ChatGPT • AI Assistant

A sequence diagram with two swim lanes: 'models.py execution' and 'services.py execution'. Timeline flows top to bottom. Step 1 (models lane): 'Python starts executing models.py'. Step 2 (models lane): 'Hits: from services import process_data'. Step 3: Arrow crosses to services lane: 'Python starts executing services.py'. Step 4 (services lane): 'Hits: from models import User'. Step 5: Arrow crosses back to models lane: 'Python checks sys.modules — models IS there, but INCOMPLETE. User class not yet defined.' Step 6 (models lane, red box): 'ImportError: cannot import name User from partially initialized module models'. On the right, a fix flow diagram: 'Strategy 1: Extract User base to base.py (no cycle)'. 'Strategy 2: Move import inside method (defer until runtime)'. 'Strategy 3: if TYPE_CHECKING: (skip at runtime)'. Each strategy shown with a small before/after arrow. Learning objective: Understand WHY the error occurs (partial state in sys.modules) and see all fix strategies at a glance.

Circular Import Failure — Partial Initialization Diagram

My colleague mentioned 'namespace packages' and 'circular imports.' I've hit circular import errors before and had no idea how to fix them. Can we cover both?

ChatGPT • AI Assistant

CONCEPT CARD

Two distinct topics — let me tackle them separately.

Namespace packages (PEP 420, Python 3.3+) are packages WITHOUT `__init__.py`. Python 3 can treat any directory as a package even without the init file. This enables splitting a single logical package across multiple directories or repositories. You rarely create them intentionally — but you will encounter them, and misunderstanding them causes 'why is this importable?' confusion.

Circular imports happen when module A imports module B, and module B (directly or indirectly) imports module A. Python partially executes module A, starts executing module B, hits the import of A, finds A already in `sys.modules` but PARTIALLY initialized, and gives you either an `ImportError` or an `AttributeError: module 'X' has no attribute 'Y'`.

EXAMPLE CARD

Example 1 — Basic: Namespace packages — what they look like

code — Code Example

```
# A namespace package — NO __init__.py anywhere
company_tools/          ← NOT a regular package — no __init__.py
  logging/              ← also no __init__.py
    formatter.py
  database/
    connector.py

# Python 3 can still import from this:
import company_tools.logging.formatter # Works as namespace package
from company_tools.database import connector

# Where you encounter this:
# 1. Split packages: same top-level name in two different repos/paths
```

```
# 2. Legacy code migrated from Python 2
# 3. Some test frameworks that inject test paths
```

Student

So I can have two directories with the same name on `sys.path` and Python merges them?

ChatGPT • AI Assistant

Exactly — that's the key use case for namespace packages. Two repos can both have a `company/` directory and Python merges them into a single `company` namespace:

code — Code Example

```
# repo1/company/auth.py
# repo2/company/payments.py
# sys.path = ['repo1/', 'repo2/']

from company import auth      # Finds repo1/company/auth.py
from company import payments  # Finds repo2/company/payments.py
# Both work! Python merges the two 'company' directories into one
namespace.
# This would FAIL if either had __init__.py — regular packages don't
merge.
```

EXAMPLE CARD**Example 2 — Intermediate: Understanding circular imports**

python — Code Example

```
# models.py
from services import process_data    # Import from services.py

class User:
    def save(self):
        process_data(self)

# services.py
from models import User              # Import from models.py

def process_data(user):
```

```
if isinstance(user, User):
    print(f'Processing {user}')
```

```
# What happens when you run: python models.py
# 1. Python starts executing models.py
# 2. Line 1: 'from services import process_data' → start executing
services.py
# 3. In services.py line 1: 'from models import User' → models.py is
in sys.modules
# BUT it's only HALF executed (only up to line 1 ran)
# 4. User class doesn't exist yet in the partially-executed models
module!
# ImportError: cannot import name 'User' from partially initialized
module 'models'
```

Student

How do I fix a circular import?

ChatGPT • AI Assistant

CONCEPT CARD

There are four strategies, in order of preference:

1. **Restructure** — extract the shared code into a third module that neither A nor B imports from each other
2. **Move the import** — import inside the function/method instead of at the module top level (deferred import)
3. **Use TYPE_CHECKING** — import only for type hints, not at runtime
4. **Reorganize** — the circular import usually signals a design problem; fix the architecture

EXAMPLE CARD

Example 3 — Practical: Fixing circular imports — all strategies

python — Code Example

```
# — STRATEGY 1: Extract shared code —————
# Create a third module for shared types/base classes

# base.py (new file — no imports from models or services)
class UserBase:
```

```

    pass

# models.py (imports from base, not from services)
from base import UserBase
class User(UserBase):
    def save(self): ...

# services.py (imports from base, not from models)
from base import UserBase
def process_data(user: UserBase):
    print(f'Processing {user}')

# — STRATEGY 2: Deferred (local) import —————
# models.py
class User:
    def save(self):
        from services import process_data # Import INSIDE the method!
        process_data(self)
        # By the time save() is called, services.py is fully loaded

# services.py
from models import User # This is now fine – models.py is fully
                        loaded first

def process_data(user):
    if isinstance(user, User):
        print(f'Processing {user}')

```

python – Code Example

```

# — STRATEGY 3: TYPE_CHECKING guard —————
# Use this when you need the import ONLY for type hints (mypy/pyright)

# services.py
from __future__ import annotations # Makes all type hints strings
(lazy)
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from models import User # Only imported when type checker runs
                           # NOT imported at runtime – no circular
                           dependency

def process_data(user: 'User') → None: # 'User' is a string at

```

```
runtime
print(f'Processing {user}')
```

EXAMPLE CARD

Example 4 — Real Business Example: Fixing a real circular dependency in a Flask app

python — Code Example

```
# BEFORE — circular dependency:
# app.py creates Flask app, imports db from models
# models.py imports app to get the Flask app object for
db.init_app(app)

# app.py
from flask import Flask
from models import db, User          # models.py imports from app.py!
app = Flask(__name__)
db.init_app(app)

# models.py
from app import app                  # Circular! app.py imports
models.py!
from flask_sqlalchemy import SQLAlchemy
db = SQLAlchemy(app)
class User(db.Model): ...

# — AFTER — Application Factory pattern —————
# extensions.py — new file, no circular deps
from flask_sqlalchemy import SQLAlchemy
db = SQLAlchemy() # NOT bound to app yet

# models.py — imports from extensions only
from extensions import db
class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(80))

# app.py — application factory
from flask import Flask
from extensions import db
from models import User # Safe now — models.py doesn't import from
app.py

def create_app():
```

```
app = Flask(__name__)
db.init_app(app)      # Bind db to app here
return app
```

No circular dependencies!

❗ COMMON PITFALL / MISTAKE

✗ Incorrect — assuming `__init__.py` is optional and then being confused:

python – Code Example

```
# Python 3 can import from directories WITHOUT __init__.py
# as namespace packages. But regular packages with __init__.py
# behave differently – they can't merge across directories.

# If you accidentally OMIT __init__.py from a regular package:
# Python 3 still imports it (as namespace package) – no error!
# But __init__.py code won't run, __all__ won't be defined,
# and the package behaves unexpectedly. Hard to debug.
```

✓ Correct — always create `__init__.py` intentionally:

python – Code Example

```
# For regular packages: always include __init__.py
# (Even if it's empty – it makes intent explicit)
# For namespace packages: intentionally OMIT it and document why
```

📖 INTERVIEW CARD

- **Frequency:** High — circular imports are a classic interview and real-world problem
- **Question:** 'You have a circular import between `models.py` and `services.py`. What causes it and how do you fix it?'
- **Expected Answer:** Circular imports occur when A imports B and B imports A. Python partially executes A, then B, finds A in `sys.modules` but incomplete, causing `ImportError` or missing attributes. Fixes: (1) extract shared code to a third module, (2) defer import to inside the function that needs it, (3) use `TYPE_CHECKING` for type-hint-only imports, (4) restructure with application factory pattern.
- **Follow-up:** 'What is a namespace package and how does it differ from a regular package?'

🔮 AI DEEP DIVE PROMPT

Copy this into Claude or ChatGPT:

"I understand Python's module system. Now teach me advanced topics: importlib for dynamic imports, writing custom import hooks, lazy loading with importlib.util.LazyLoader, the **import** function, pkgutil for iterating packages, namespace packages across repos, how pytest manipulates sys.path, and how tools like Poetry/setuptools define packages. For each: (1) explain the real problem it solves, (2) show a minimal working example, (3) give one interview or scenario question. Challenge me before showing answers."

🔥 EXERCISE

Fix this circular import chain. Two files import each other and the project won't start:

python – Code Example

```
# order.py
from customer import Customer

class Order:
    def __init__(self, customer_id, total):
        self.customer = Customer.get(customer_id)
        self.total = total

    def invoice_address(self):
        return self.customer.address

# customer.py
from order import Order

class Customer:
    def __init__(self, customer_id, name, address):
        self.customer_id = customer_id
        self.name = name
        self.address = address

    @staticmethod
    def get(customer_id):
        return Customer(customer_id, 'Alice', '123 Main St')

    def all_orders(self):
        return Order.get_by_customer(self.customer_id)
```

Apply at least TWO different fix strategies and explain the tradeoff of each.

🔗 PRACTICE QUIZ

Concept Questions:

1. What is the key difference between a regular package and a namespace package?
2. Why does Python show 'partially initialized module' in a circular import error?
3. Why is the `TYPE_CHECKING` strategy for circular imports safe at runtime?

Output Prediction Questions:

1. Module A imports B at the top level. Module B imports A at the top level. You run `python A.py`. At what point in B's execution does the error occur, and what does `sys.modules['A']` contain at that moment?
2. Two directories `repo1/utils/` and `repo2/utils/` are both on `sys.path` and neither has `__init__.py`. You do `from utils import foo`. Where does Python look first?

Debugging Question:

1. A developer uses strategy 2 (local import) but still gets a circular import error:

python – Code Example

```
# models.py
class User:
    def save(self):
        from services import process_data
        process_data(self)

# services.py
from models import User # Still circular!
from models import save_all # Also needs this

def process_data(user):
    pass

def save_all(users):
    for u in users:
        u.save()
```

Where is the remaining circular dependency and how do you eliminate it?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. A regular package requires `__init__.py` . Its directory is ONLY searchable as a single package on one `sys.path` entry. A namespace package has no `__init__.py` — Python 3 can merge multiple directories with the same name from different `sys.path` entries into one importable namespace.
2. When Python starts importing module A and encounters `import B` which imports A again, A is already in `sys.modules` — but only partially executed (only the lines before the circular import ran). Python returns this incomplete module object, and any attributes not yet defined cause `AttributeError` or `ImportError` .
3. `typing.TYPE_CHECKING` is `False` at runtime — the `if TYPE_CHECKING:` block is skipped entirely during normal execution. Only type checkers (mypy, pyright) set it to `True` when analyzing code. So the circular import never actually happens at runtime.
4. The error occurs at `from models import User` inside `services.py` . At that moment, `sys.modules['A']` (models) contains the partially initialized module object — the `User` class has not been defined yet because execution of `models.py` stopped at line 1 (`from services import process_data`).
5. Python searches `repo1/utils/` first (whichever `sys.path` entry appears first). If `foo` is found there, it returns it. Namespace packages are merged only when a name is NOT found in the first location.
6. The remaining circular dependency: `services.py` imports `save_all` from `models.py` at the top level — but `save_all` is defined in `services.py` itself. Wait, actually the bug is `from models import save_all` in `services.py` — if `save_all` is in `models.py` , this creates a circular import even with the local import in `User.save()` . Fix: move `save_all` to a third module (e.g., `batch_ops.py`) that neither `models` nor `services` imports from, breaking the cycle.

SUMMARY CARD

- A module is a `.py` file; a package is a directory with `__init__.py` ; a namespace package is a directory WITHOUT `__init__.py`
- `sys.path` is searched in order — script directory is [0], so local files shadow stdlib
- `sys.modules` caches every import — the same module is only executed once

- `__init__.py` runs on first package import — keep it lean with only public API re-exports and `__all__`
- `__all__` controls `from package import *` and signals the public API contract
- Prefer absolute imports (PEP 8) — use relative imports for intra-package sibling references
- Relative imports require `__package__` to be set — they fail in directly-run scripts
- Circular imports: extract shared code to a third module, defer imports to function scope, or use `TYPE_CHECKING`
- The Application Factory pattern is the standard fix for circular imports in Flask/Django

🔍 CHECKPOINT QUESTION

You are designing a large Python package `analytics/`. A teammate says: 'I'll put all the imports and business logic directly in `__init__.py` so users only need one import.' What are three specific problems this will cause in a production project?

🌟 CHAPTER REVIEW & SUMMARY

What You Learned:

Understanding of the core concepts presented in this chapter including modules and packages, their architectural components, practical use cases, and design paradigms.

Quick Revision Notes:

- Key definitions and nomenclature for the chapter topic.
- Hands-on code execution patterns and setup checklists.
- Analogy-driven visualization of data structures or layered setups.

Core Visualized Components:

- **Python Import System — Full Lookup Pipeline:** A vertical flowchart showing Python's import resolution pipeline. Starting node:...